

Session13_13.07.2026

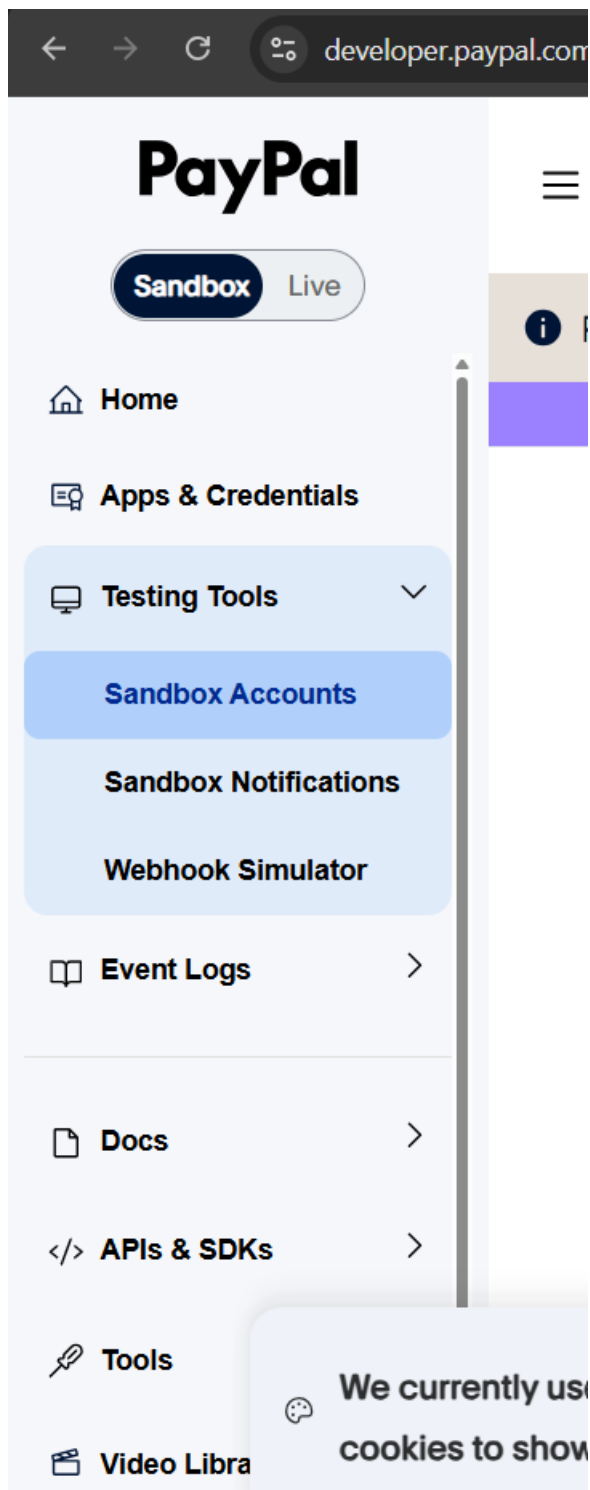
Lab Tasks

1. Task 1: Sandbox Accounts

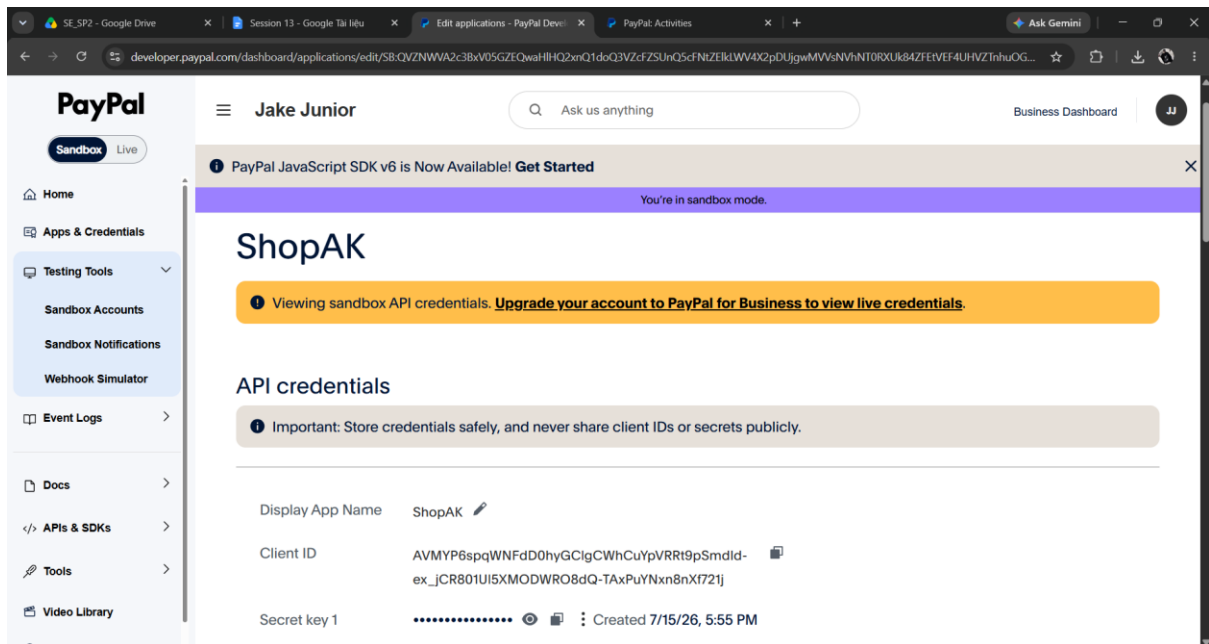
Create Stripe account and enable Test Mode; add `STRIPE_SECRET_KEY` to `.env`.

Create PayPal Developer account, sandbox Business & Personal accounts, get client ID/secret.developers.

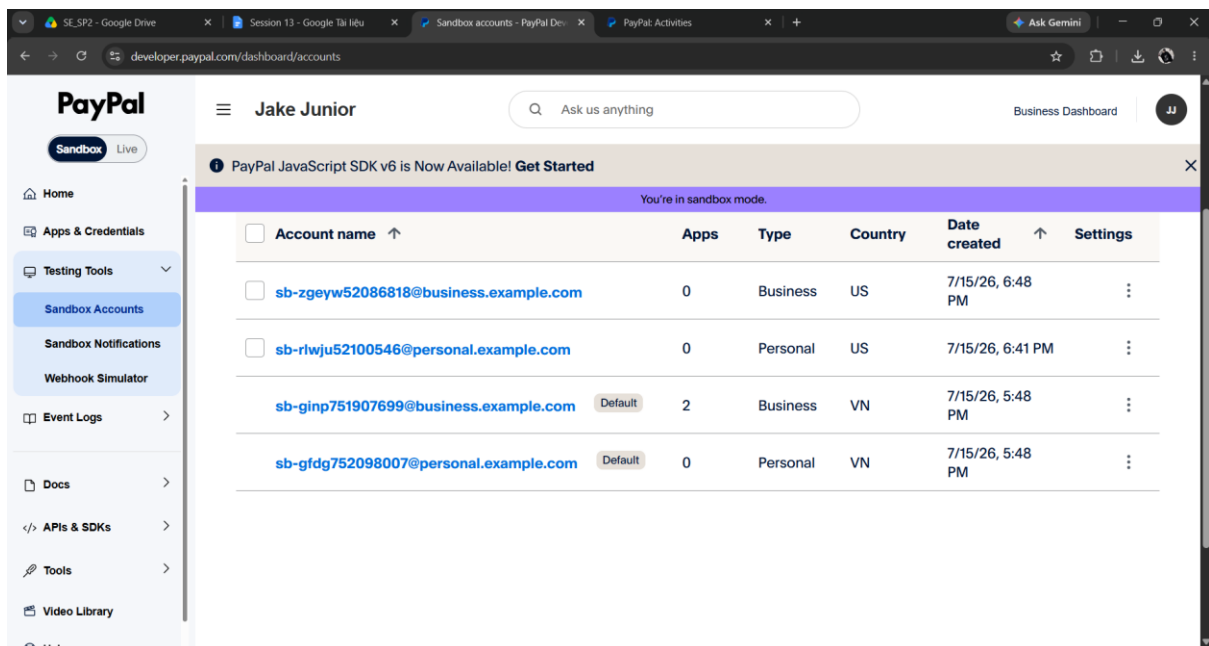
Configure VNPay sandbox settings (TmCode, secret, return URL).



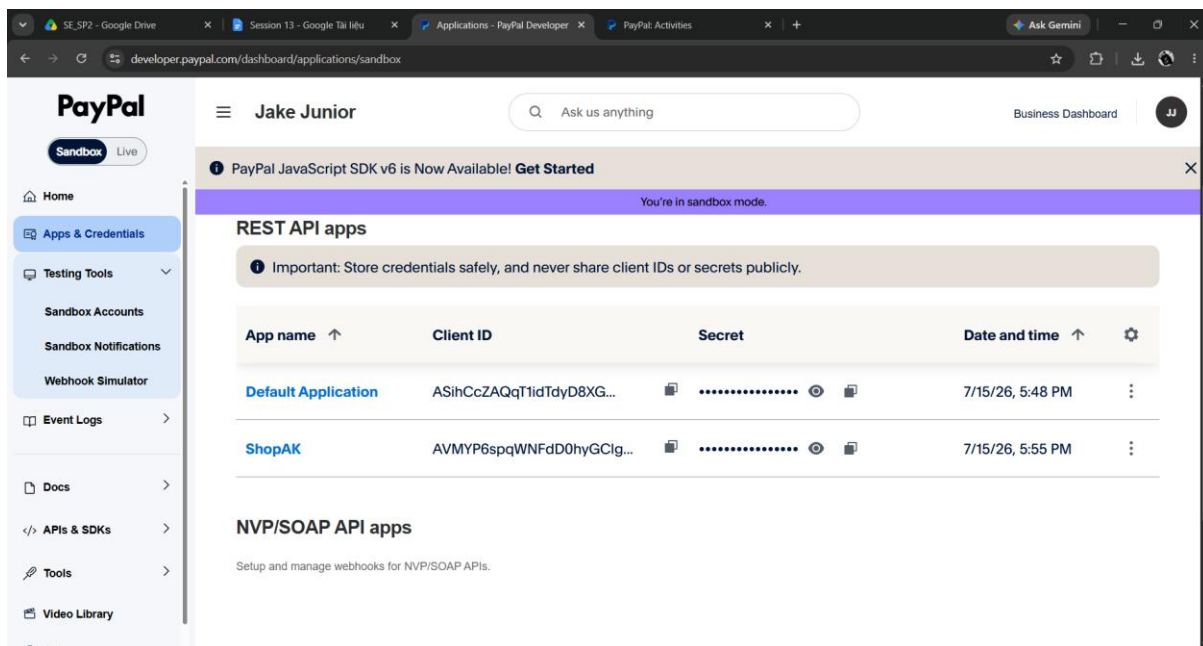
Trang **Developers** với **API keys** của Stripe



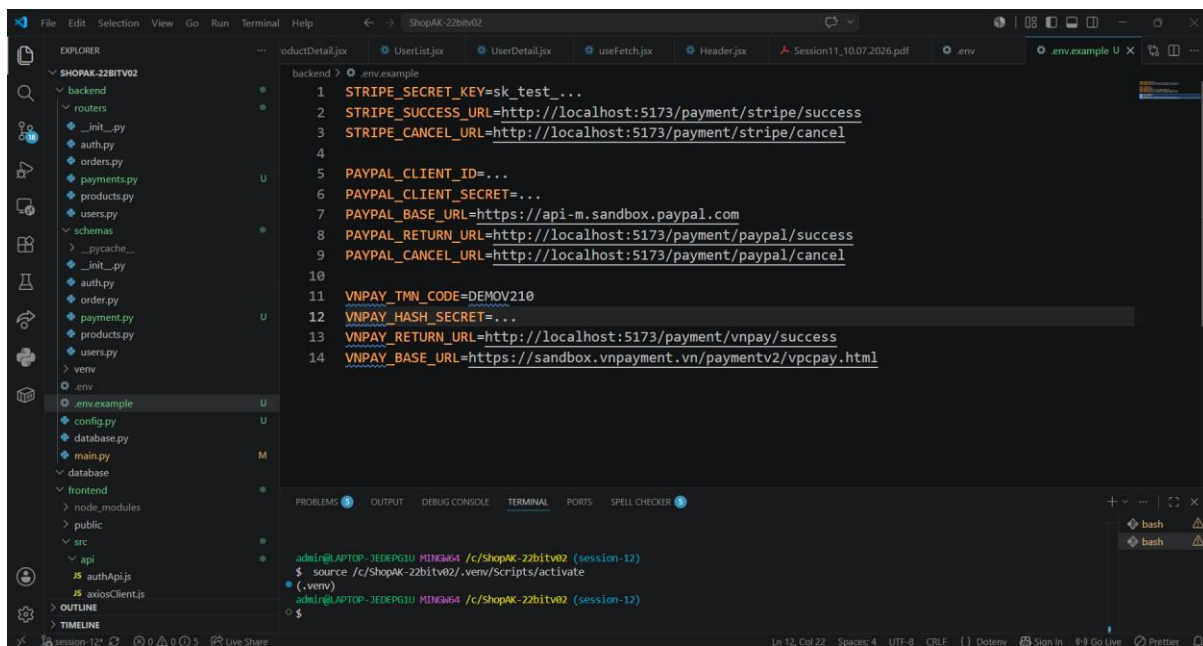
PayPal Developer Dashboard chúng ta sẽ thấy cả 2 tài khoản Business và Personal



Apps & Credentials và Client ID/Secret



File backend/.env.example



2. Task 2: Backend Payment Router

Create `payments.py` router with endpoints:

- POST `/payments/stripe/create-session`
- POST `/payments/paypal/create-order`
- POST `/payments/vnpay/create-url`
- POST `/payments/confirm`

Integrate with `OrderDB`:

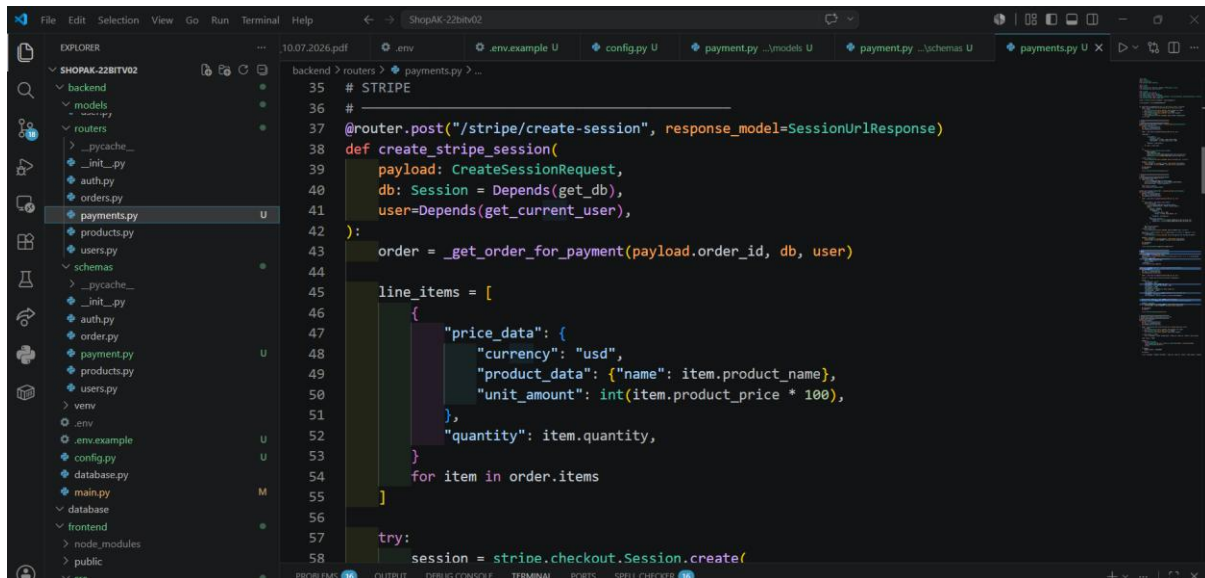
- Validate order exists and belongs to current user.
- Make sure only orders with `status != PAID` can be paid.

Implement simple confirm logic:

- g. `confirm` sets `order.status = "PAID"` and returns updated order.

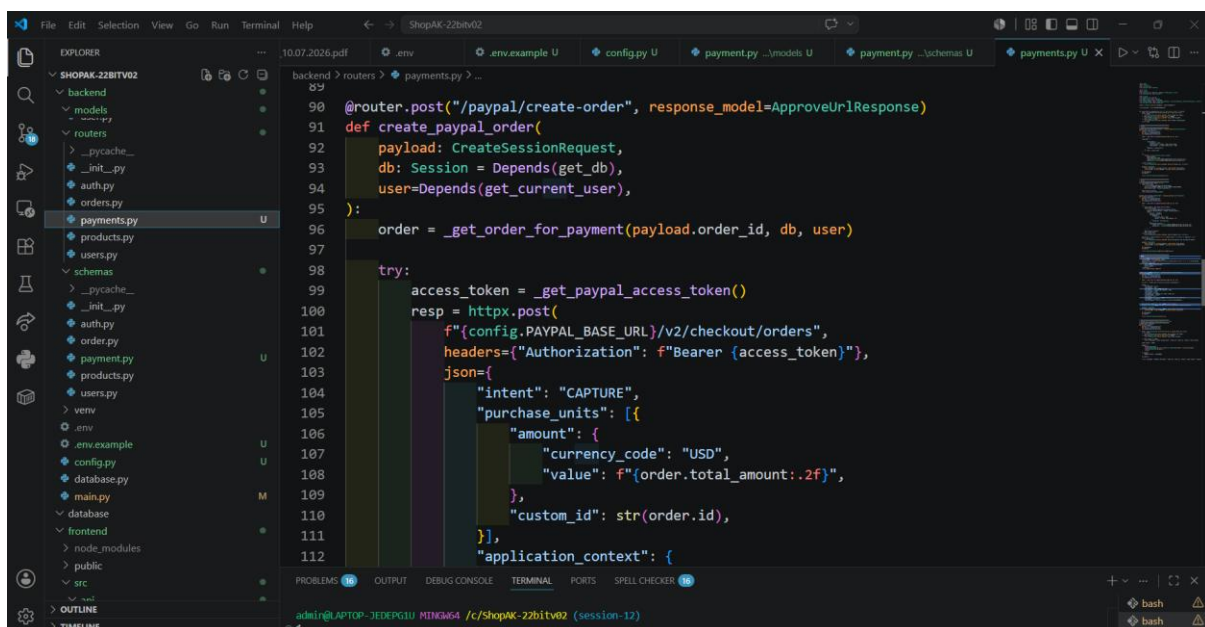
File code backend/routers/payments.py gồm 4 đoạn:

- `create_stripe_session`



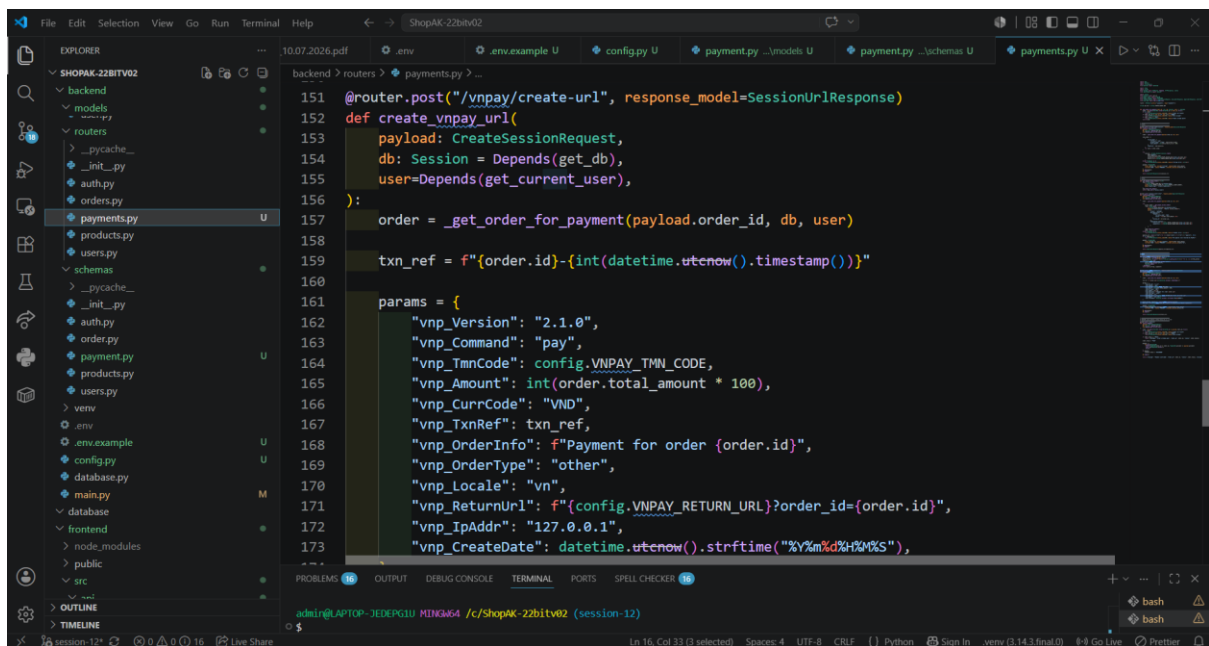
```
35 # STRIPE
36 #
37 @router.post("/stripe/create-session", response_model=SessionUrlResponse)
38 def create_stripe_session(
39     payload: CreateSessionRequest,
40     db: Session = Depends(get_db),
41     user=Depends(get_current_user),
42 ):
43     order = _get_order_for_payment(payload.order_id, db, user)
44
45     line_items = [
46         {
47             "price_data": {
48                 "currency": "usd",
49                 "product_data": {"name": item.product_name},
50                 "unit_amount": int(item.product_price * 100),
51             },
52             "quantity": item.quantity,
53         }
54         for item in order.items
55     ]
56
57     try:
58         session = stripe.checkout.Session.create(
```

- `create_paypal_order`



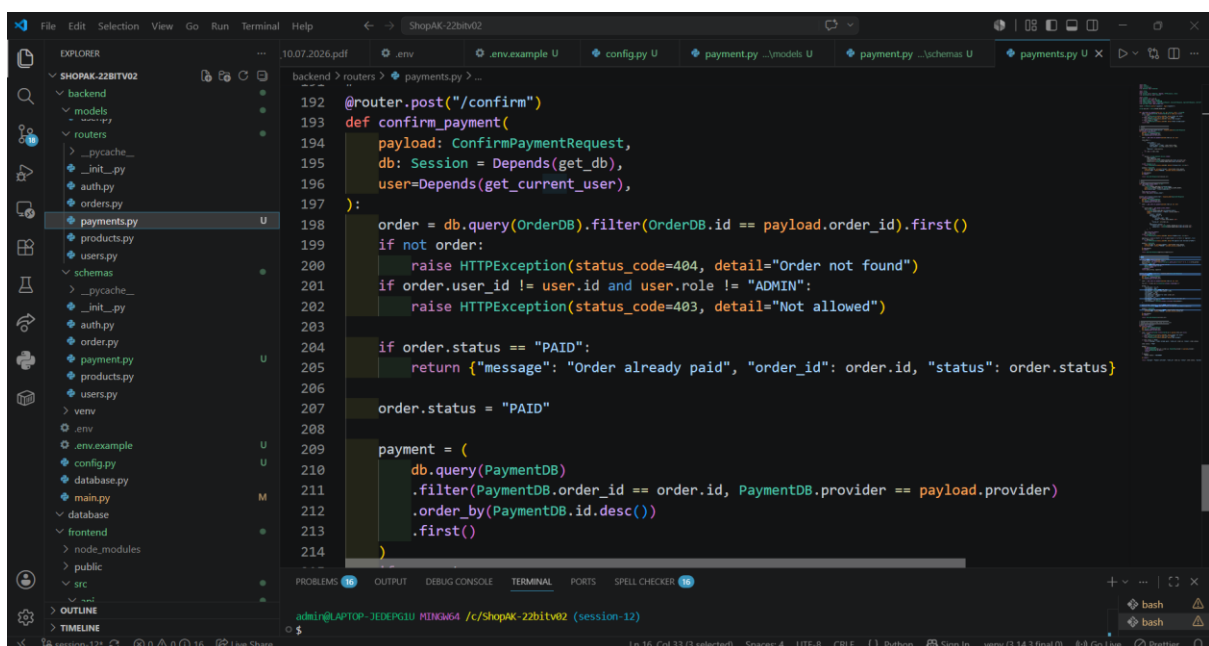
```
89
90 @router.post("/paypal/create-order", response_model=ApproveUrlResponse)
91 def create_paypal_order(
92     payload: CreateSessionRequest,
93     db: Session = Depends(get_db),
94     user=Depends(get_current_user),
95 ):
96     order = _get_order_for_payment(payload.order_id, db, user)
97
98     try:
99         access_token = _get_paypal_access_token()
100         resp = httpx.post(
101             f"{config.PAYPAL_BASE_URL}/v2/checkout/orders",
102             headers={"Authorization": f"Bearer {access_token}"},
103             json={
104                 "intent": "CAPTURE",
105                 "purchase_units": [{
106                     "amount": {
107                         "currency_code": "USD",
108                         "value": f"{order.total_amount:.2f}",
109                     },
110                     "custom_id": str(order.id),
111                 }],
112                 "application_context": {
```

- `create_vnpay_url`



```
151 @router.post("/vnpay/create-url", response_model=SessionUrlResponse)
152 def create_vnpay_url(
153     payload: CreateSessionRequest,
154     db: Session = Depends(get_db),
155     user=Depends(get_current_user),
156 ):
157     order = _get_order_for_payment(payload.order_id, db, user)
158
159     txn_ref = f"{order.id}-{int(datetime.utcnow().timestamp())}"
160
161     params = {
162         "vnp_Version": "2.1.0",
163         "vnp_Command": "pay",
164         "vnp_TmnCode": config.VNPAY_TMN_CODE,
165         "vnp_Amount": int(order.total_amount * 100),
166         "vnp_CurrCode": "VND",
167         "vnp_TxnRef": txn_ref,
168         "vnp_OrderInfo": f"Payment for order {order.id}",
169         "vnp_OrderType": "other",
170         "vnp_Locale": "vn",
171         "vnp_ReturnUrl": f"{config.VNPAY_RETURN_URL}?order_id={order.id}",
172         "vnp_IpAddr": "127.0.0.1",
173         "vnp_CreateDate": datetime.utcnow().strftime("%Y%m%d%H%M%S"),
```

- confirm_payment



```
192 @router.post("/confirm")
193 def confirm_payment(
194     payload: ConfirmPaymentRequest,
195     db: Session = Depends(get_db),
196     user=Depends(get_current_user),
197 ):
198     order = db.query(OrderDB).filter(OrderDB.id == payload.order_id).first()
199     if not order:
200         raise HTTPException(status_code=404, detail="Order not found")
201     if order.user_id != user.id and user.role != "ADMIN":
202         raise HTTPException(status_code=403, detail="Not allowed")
203
204     if order.status == "PAID":
205         return {"message": "Order already paid", "order_id": order.id, "status": order.status}
206
207     order.status = "PAID"
208
209     payment = (
210         db.query(PaymentDB)
211         .filter(PaymentDB.order_id == order.id, PaymentDB.provider == payload.provider)
212         .order_by(PaymentDB.id.desc())
213         .first()
214     )
```

Ở (_get_order_for_payment) chúng ta thấy rõ validate order.user_id != user.id và order.status == "PAID"

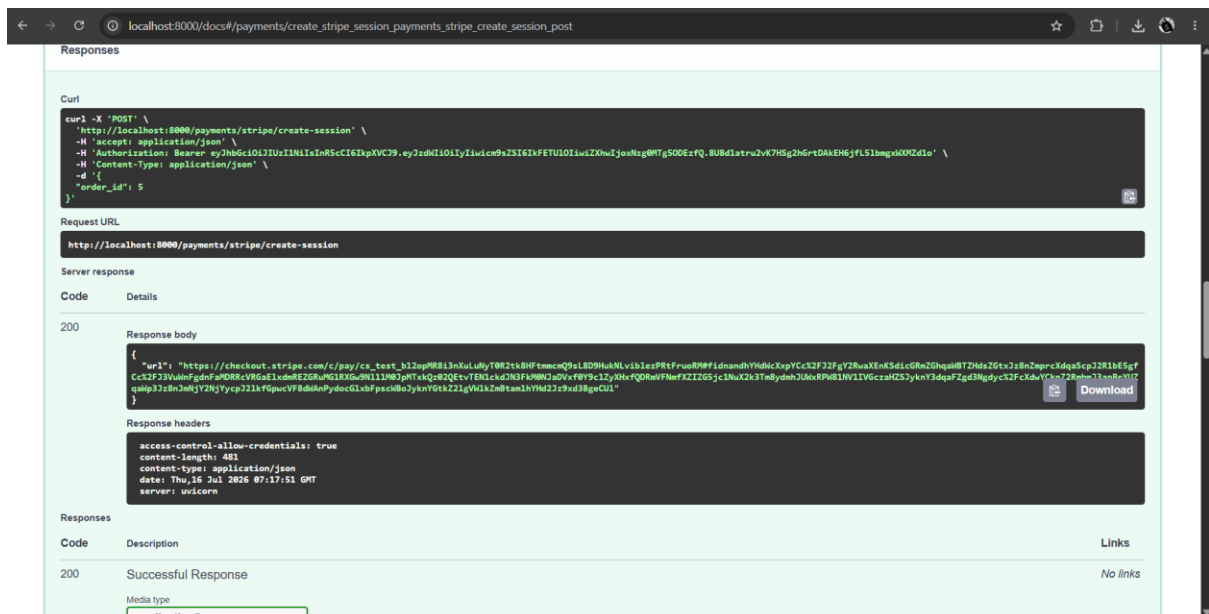
```

15 from schemas.payment import CreateSessionRequest, SessionUrlResponse, ApproveUrlResponse, Confirm
16 from auth.deps import get_current_user
17
18 @router.post("/stripe/create-session", response_model=SessionUrlResponse)
19 def _get_order_for_payment(order_id: int, db: Session, user: Any) -> OrderDB:
20     order = db.query(OrderDB).filter(OrderDB.id == order_id).first()
21     if not order:
22         raise HTTPException(status_code=404, detail="Order not found")
23     if order.user_id != user.id and user.role != "ADMIN":
24         raise HTTPException(status_code=403, detail="Not allowed")
25     if order.status == "PAID":
26         raise HTTPException(status_code=400, detail="Order already paid")
27     return order
28
29 # STRIPE
30
31 @router.post("/stripe/create-session", response_model=SessionUrlResponse)
32 def create_stripe_session(request: CreateSessionRequest, db: Session, user: Any) -> SessionUrlResponse:
33     # Implementation logic here
34
35 # STRIPE
36
37 @router.post("/stripe/create-session", response_model=SessionUrlResponse)
38 def create_stripe_session(request: CreateSessionRequest, db: Session, user: Any) -> SessionUrlResponse:
39     # Implementation logic here

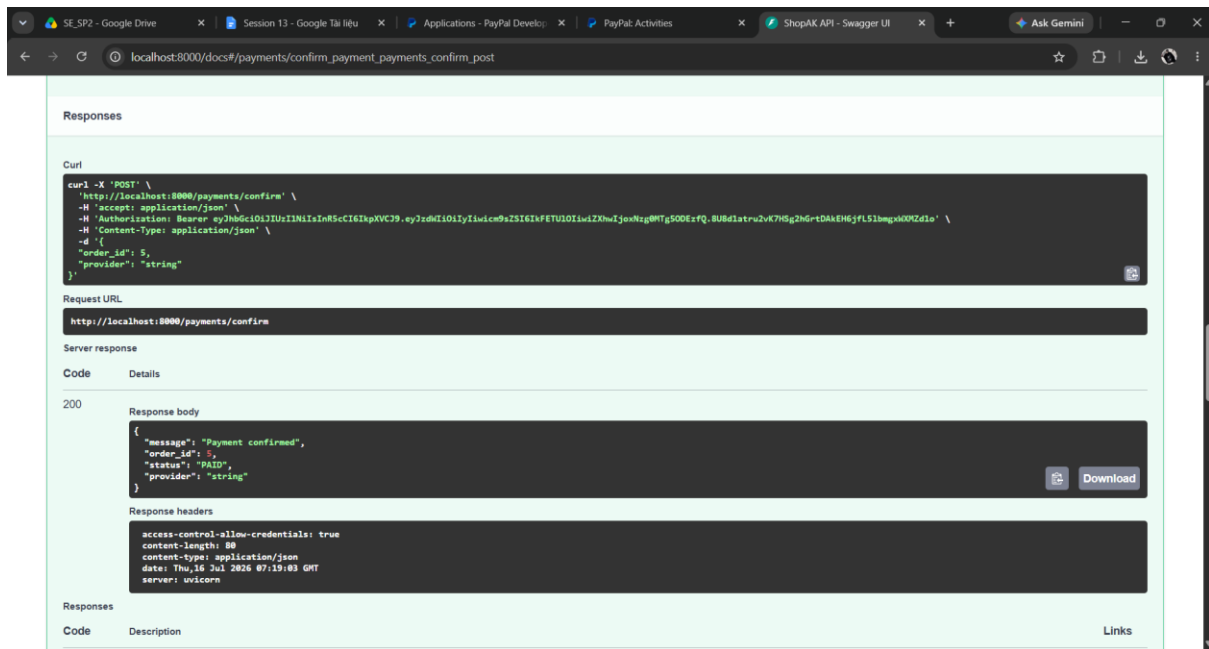
```

Test qua Swagger (<http://localhost:8000/docs>, Authorize bằng token trước):

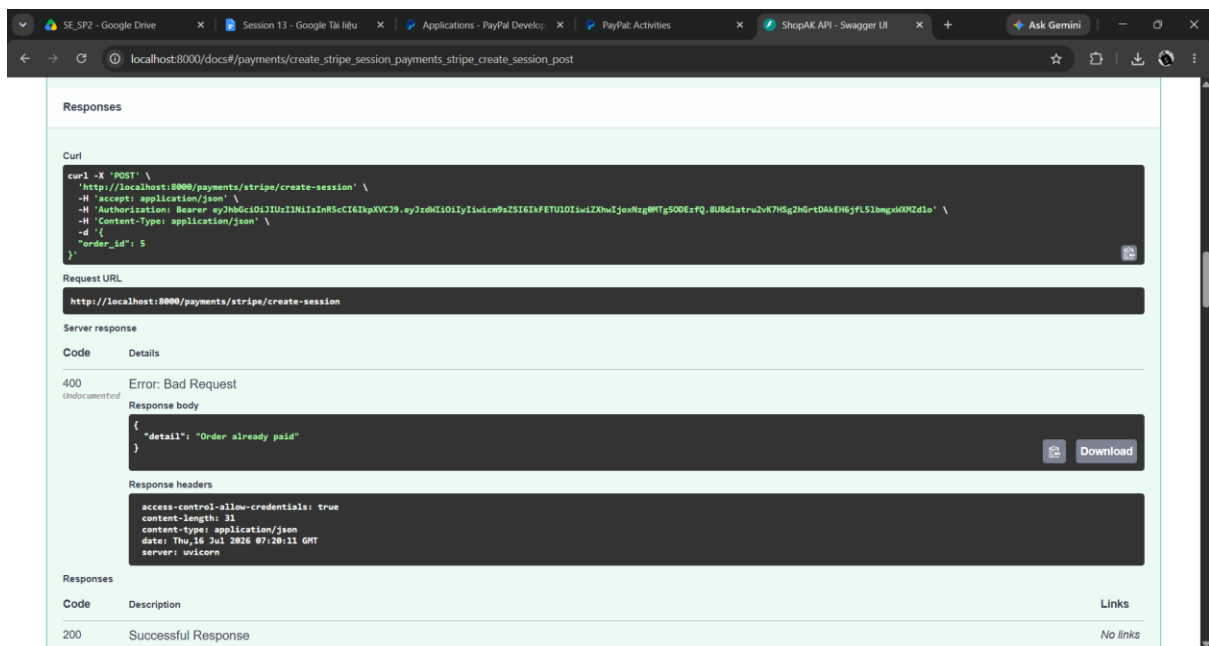
POST /payments/stripe/create-session với {"order_id": 5} sau khi excute sẽ trả về kết quả response {"url": "https://checkout.stripe.com/..."}



POST /payments/confirm với {"order_id": 5, "provider": "stripe"} sau khi excute sẽ trả về kết quả response status: "PAID"



Chúng ta sẽ thử gọi lại **POST /payments/stripe/create-session** cho đơn đã **PAID** sau khi excute sẽ trả về kết quả response lỗi **400** "Order already paid" chứng minh validation hoạt động



3. Task 3: Frontend Payment Page

File code src/pages/OrderPaymentPage.jsx

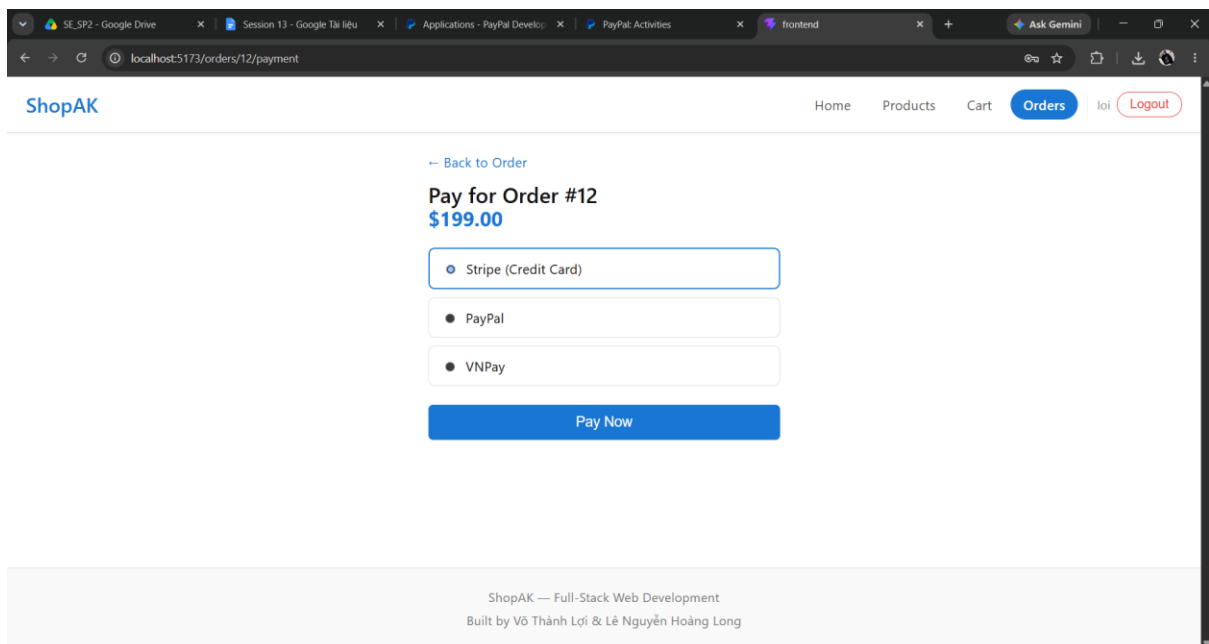
The screenshot shows a VS Code editor with the file explorer on the left displaying the project structure. The main editor shows the `OrderPaymentPage.jsx` file with the following code:

```
1 import { useEffect, useState } from "react";
2 import { useParams, Link } from "react-router-dom";
3 import { ordersApi } from "../api/ordersApi";
4 import { paymentsApi } from "../api/paymentsApi";
5
6 const OrderPaymentPage = () => {
7   const { id } = useParams();
8   const [order, setOrder] = useState(null);
9   const [loading, setLoading] = useState(true);
10  const [error, setError] = useState("");
11  const [method, setMethod] = useState("stripe");
12  const [processing, setProcessing] = useState(false);
13
14  useEffect(() => {
15    const fetchOrder = async () => {
16      setLoading(true);
17      setError("");
18      try {
19
```

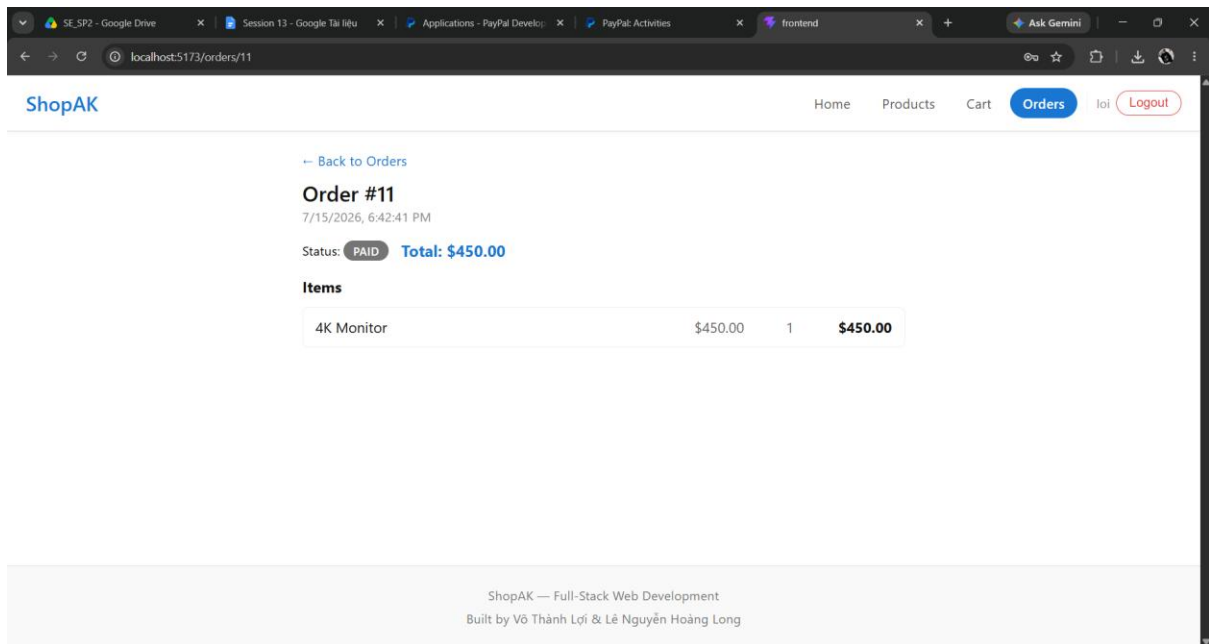
The terminal window at the bottom shows the following output:

```
(.venv)
admin@LAPTOP-JEDEPGLU MINGW64 /c/ShopAK-22bitv02/backend (session-12)
$ uvicorn main:app --reload
FROM orders
WHERE orders.id = %(id_1)s
LIMIT %(param_1)s
2026-07-16 14:28:12,364 INFO sqlalchemy.engine.Engine [cached since 140.6s ago] ['id_1': 5, 'param_1': 1]
2026-07-16 14:28:12,371 INFO sqlalchemy.engine.Engine ROLLBACK
INFO: 127.0.0.1:54031 - "POST /payments/stripe/create-session HTTP/1.1" 400 Bad Request
```

Vào trang browser: `/orders/:id/payment` ở đây chúng ta sẽ thấy đầy đủ order info với 3 radio button (Stripe/PayPal/VNPay) và nút "Pay Now"

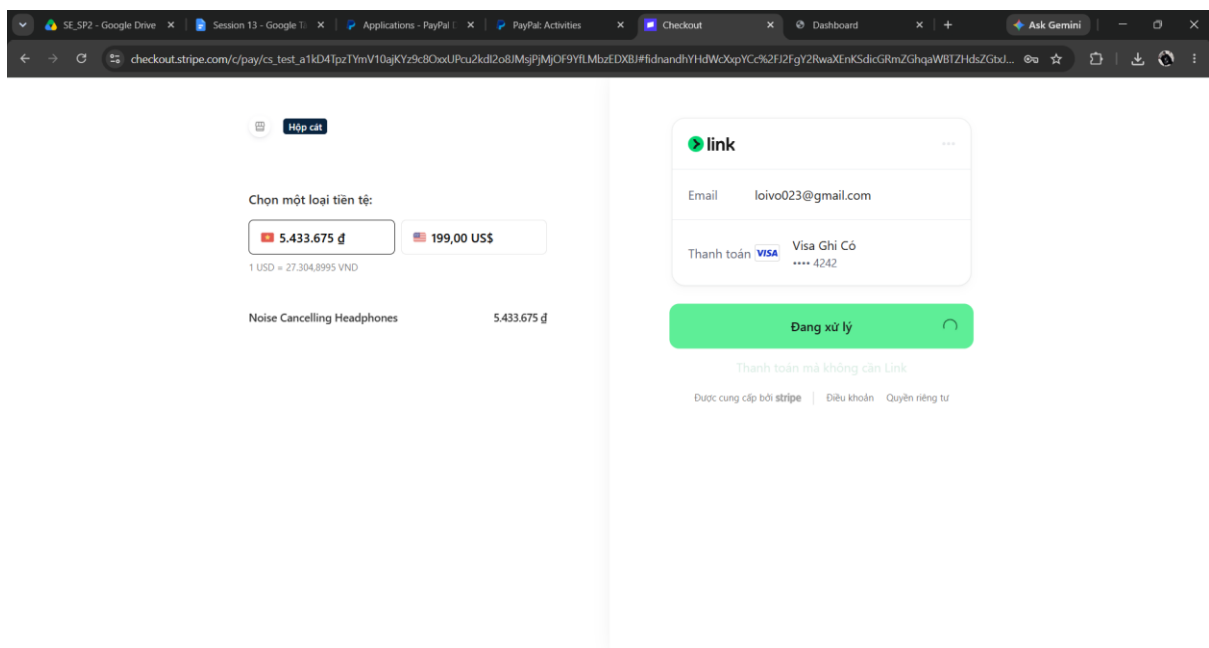


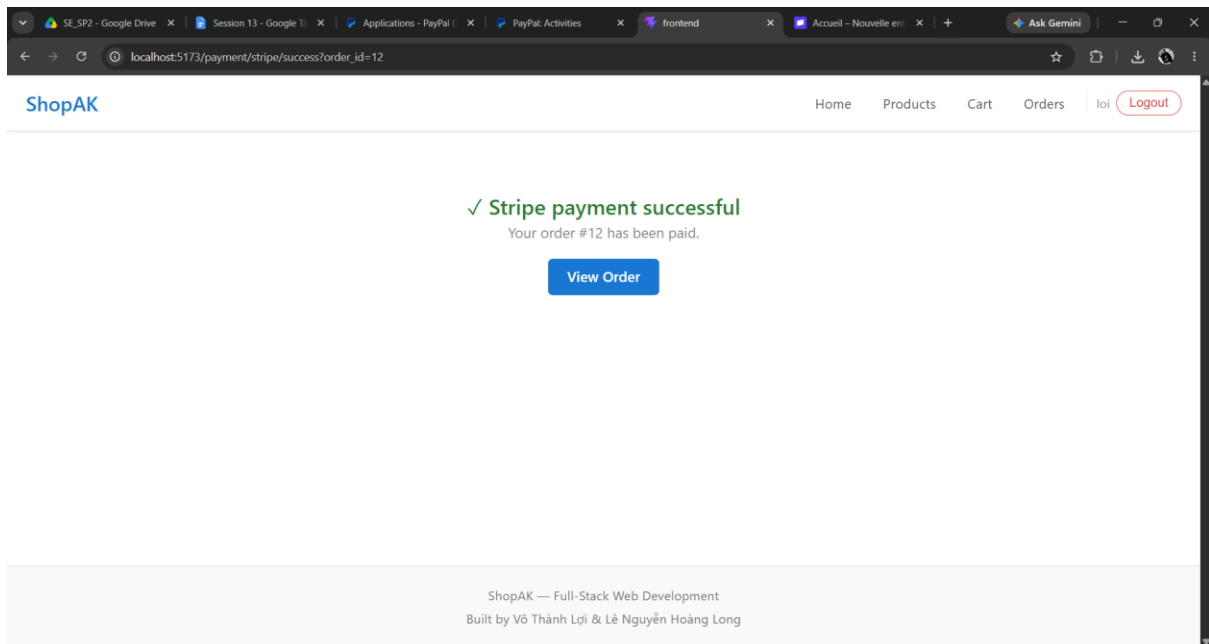
Test trường hợp đơn **đã PAID** sẽ hiện "Order Already Paid", không có form chọn phương thức



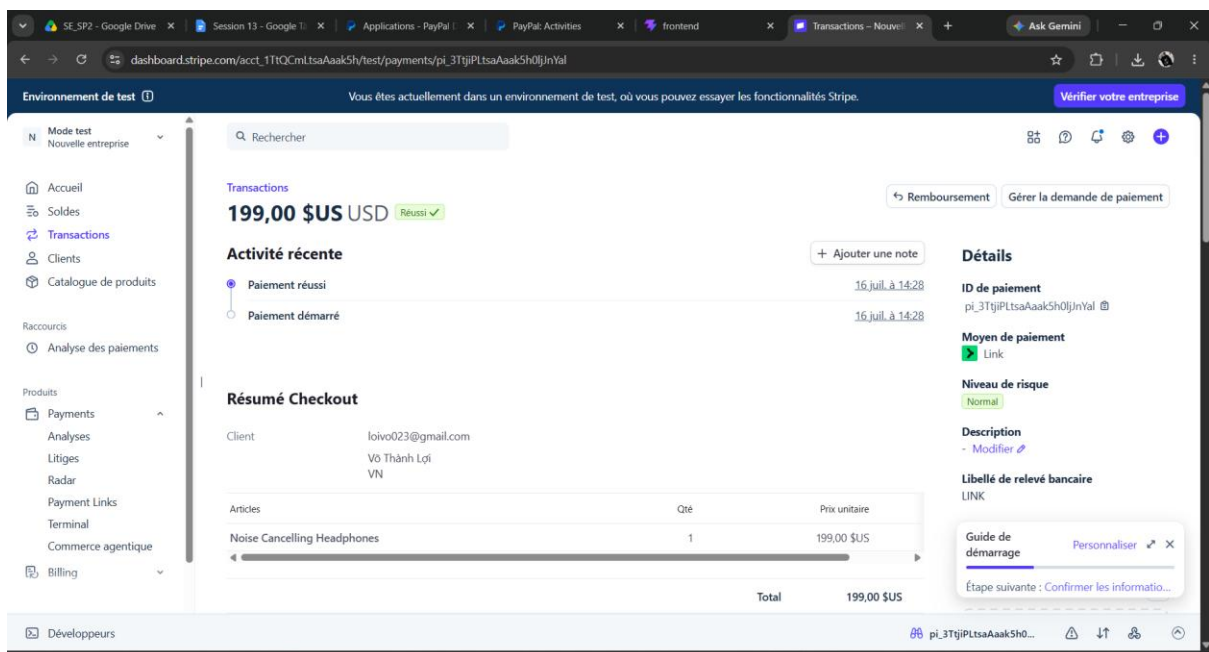
4. Task 4: Success/Cancel Pages

Test Success payment

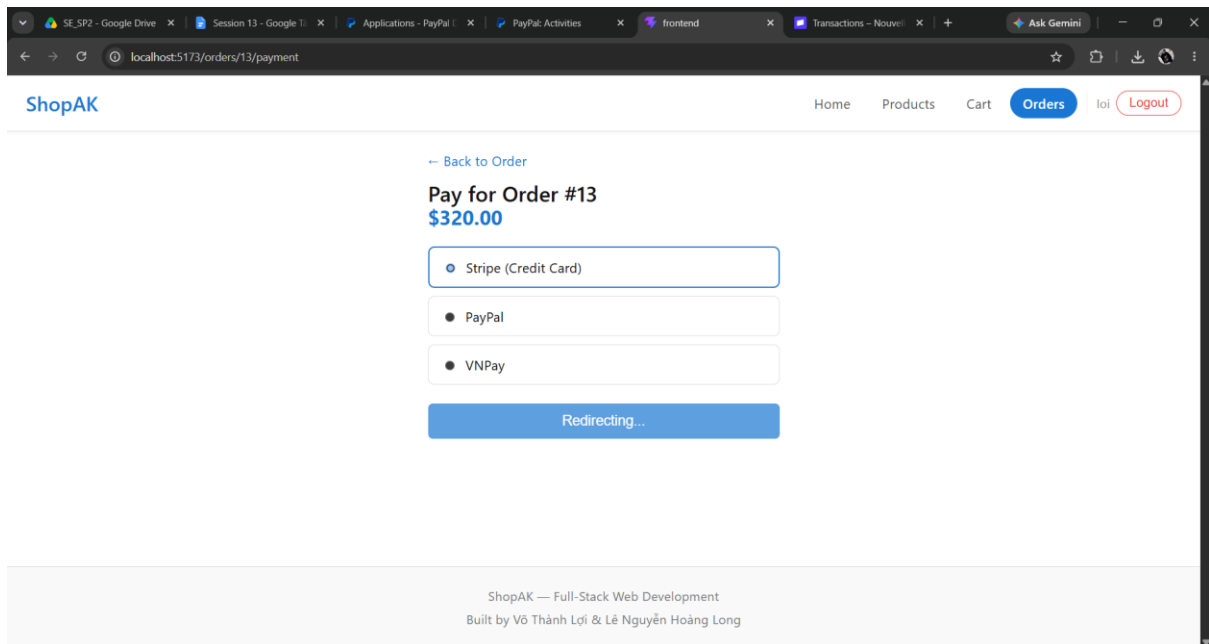




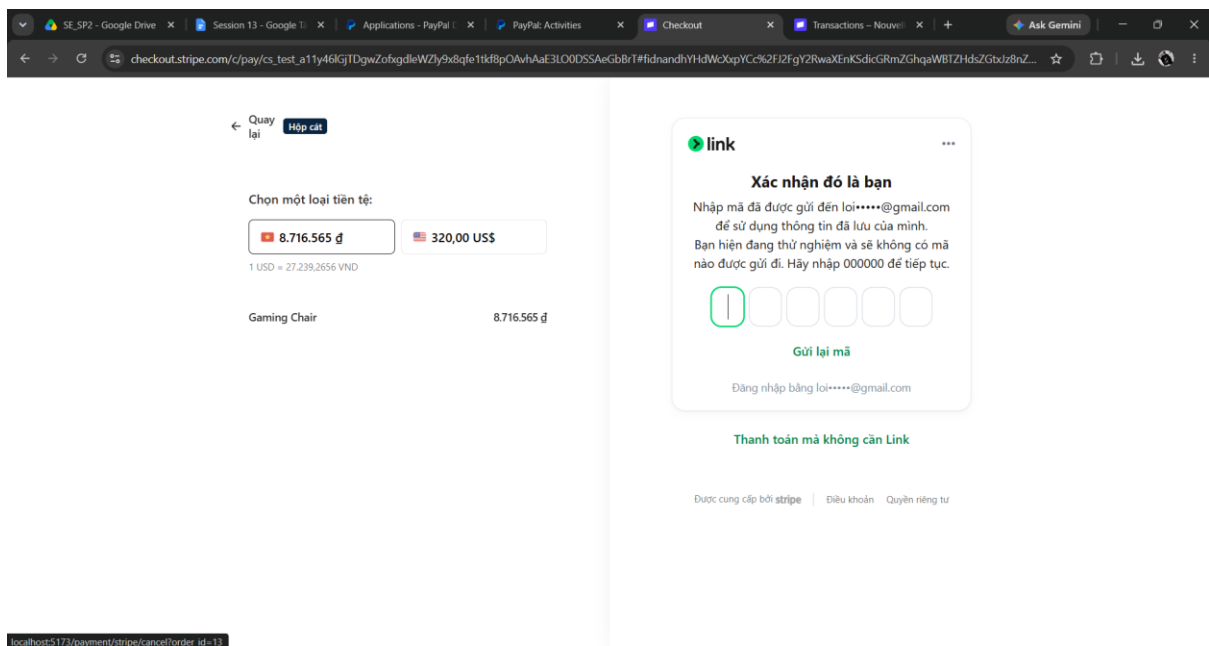
Trả về Transaction History

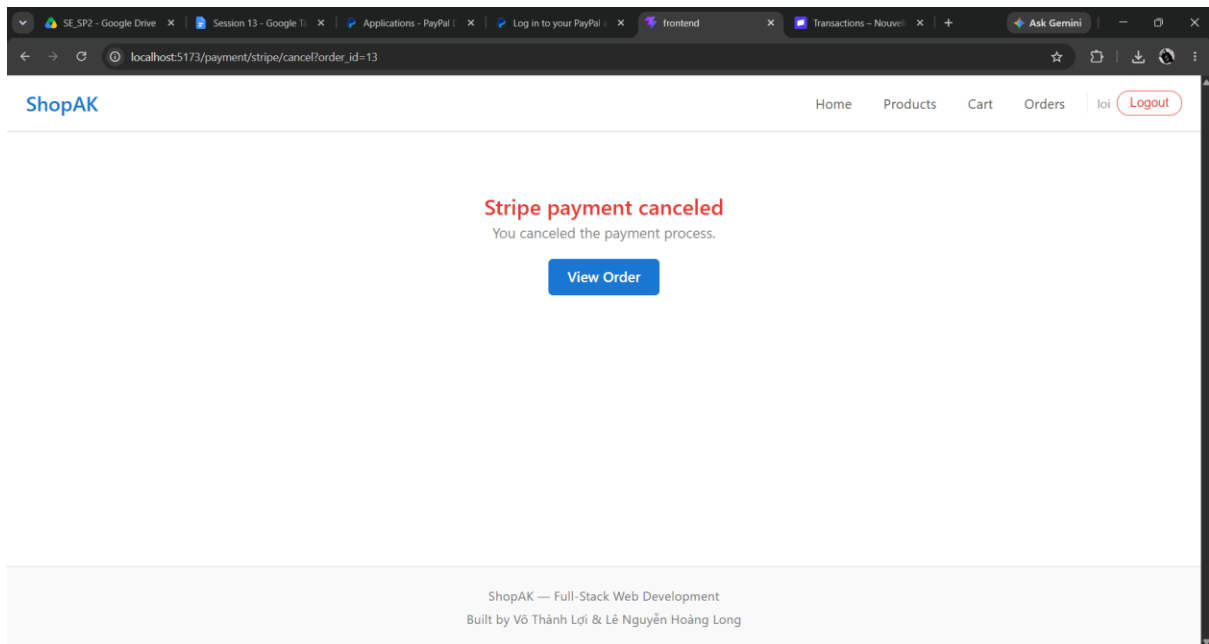


Test pages cancel

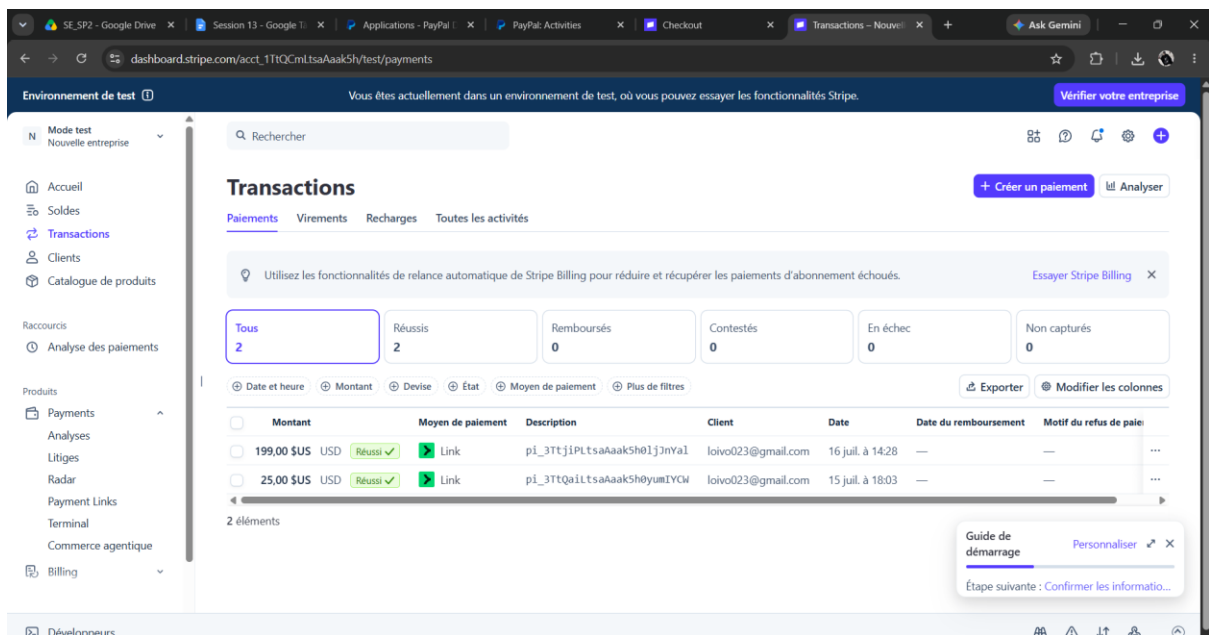


Chọn quay lại và cancel payment





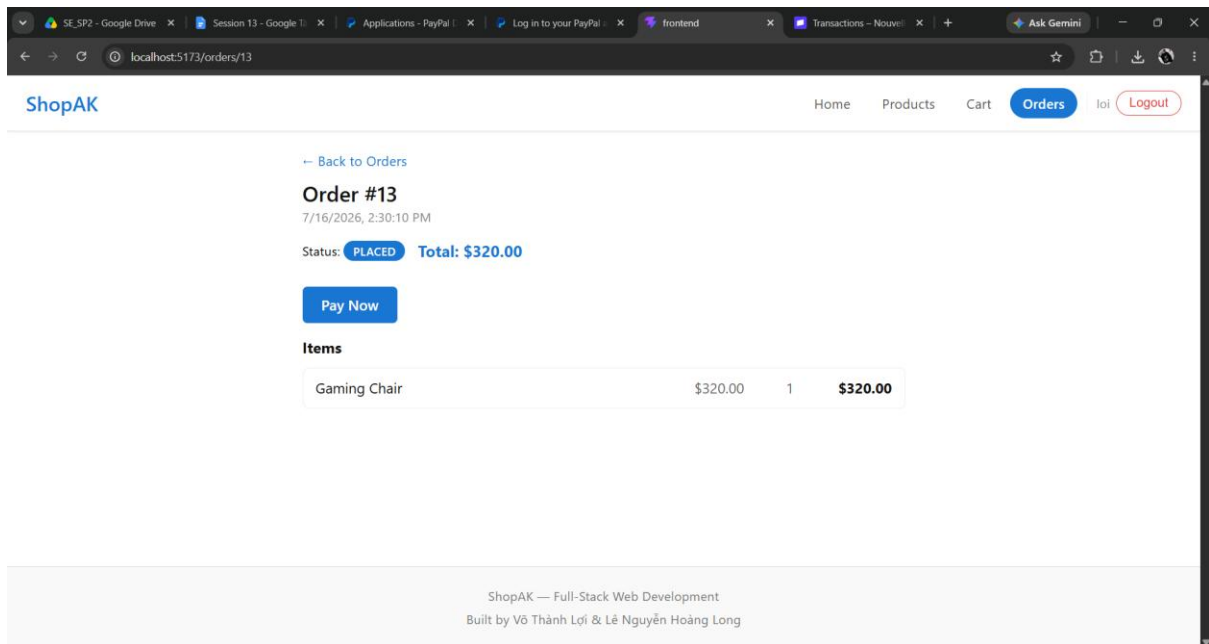
Do chưa thanh toán sẽ chưa nhận Transaction history



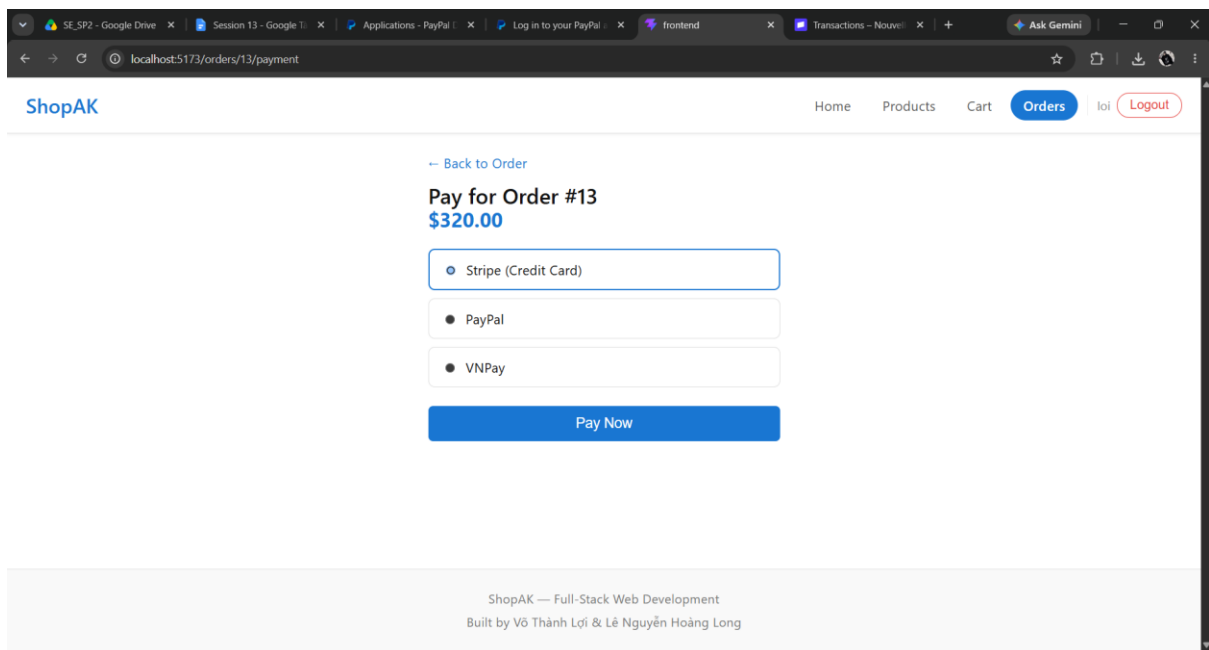
5. Task 5: End-to-End Test Scenarios

Scenario 1 - Stripe

1. Create order from cart (Order #13)



chọn Stripe và nhấn Pay Now



trang thanh toán Stripe thật và điền số thẻ 4242 4242 4242 4242, ngày bất kỳ tương lai, CVC bất kỳ

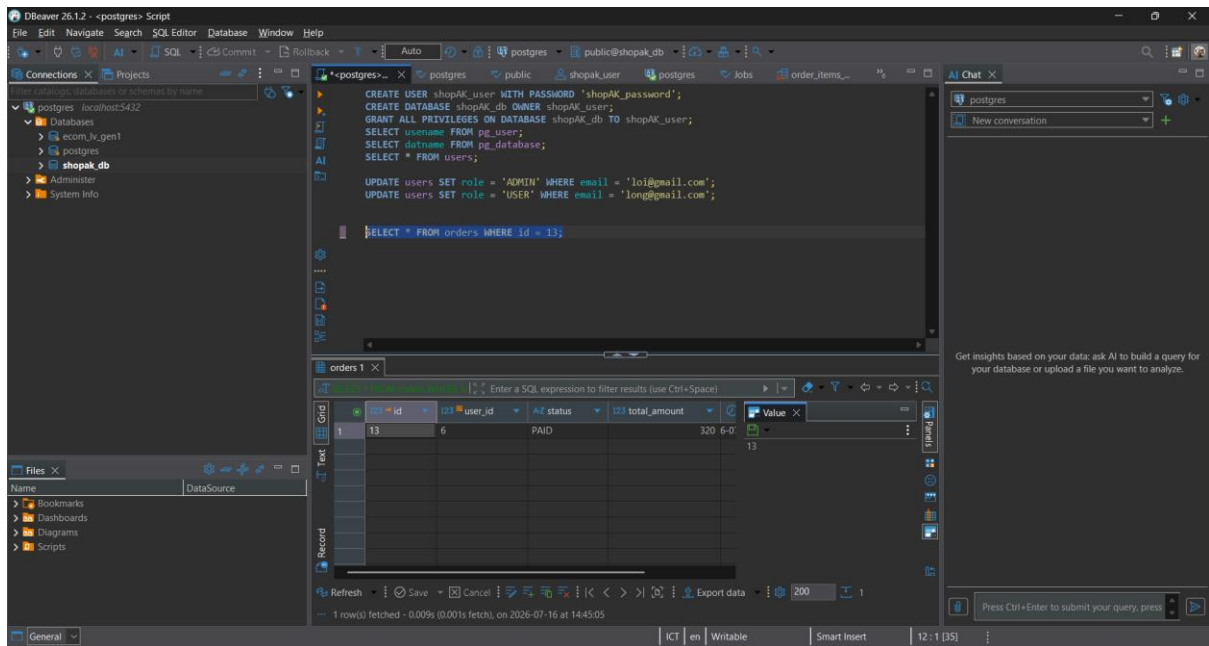
Stripe checkout page showing the payment process. The page is in Vietnamese and displays the following information:

- Chọn một loại tiền tệ:** 8.716.565 đ (Vietnamese Dong) and 320,00 US\$ (US Dollars).
- Thông tin liên hệ:** Email: loivo023@gmail.com.
- Phương thức thanh toán:** Thẻ (Card) - Visa.
- Thông tin thẻ:** 4242 4242 4242 4242, 07 / 28, 123.
- Tên chủ thẻ:** Võ Thành Lợi.
- Quốc gia hoặc khu vực:** Việt Nam.
- Thanh toán:** Button to complete the payment.

Nhấn thanh toán và lúc thanh toán thành công sẽ redirect về success page

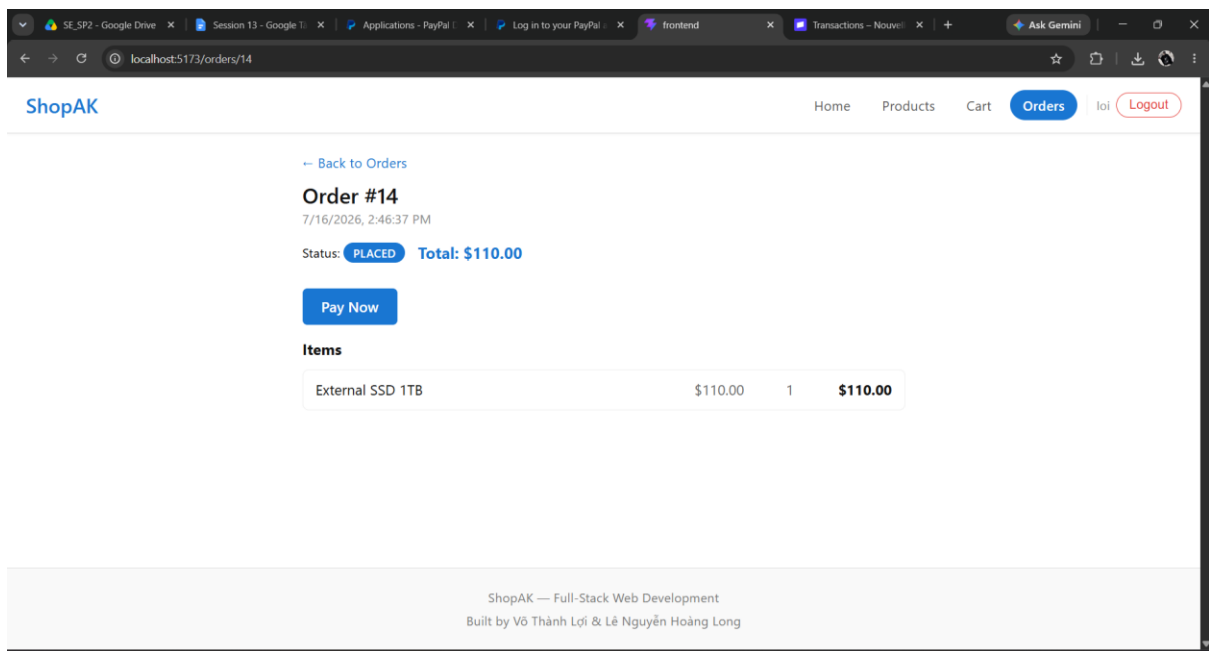
Stripe success page showing the payment confirmation. The page is in Vietnamese and displays the following information:

- Stripe payment successful:** Your order #13 has been paid.
- View Order:** Button to view the order details.
- Footer:** ShopAK — Full-Stack Web Development. Built by Võ Thành Lợi & Lê Nguyễn Hoàng Long.

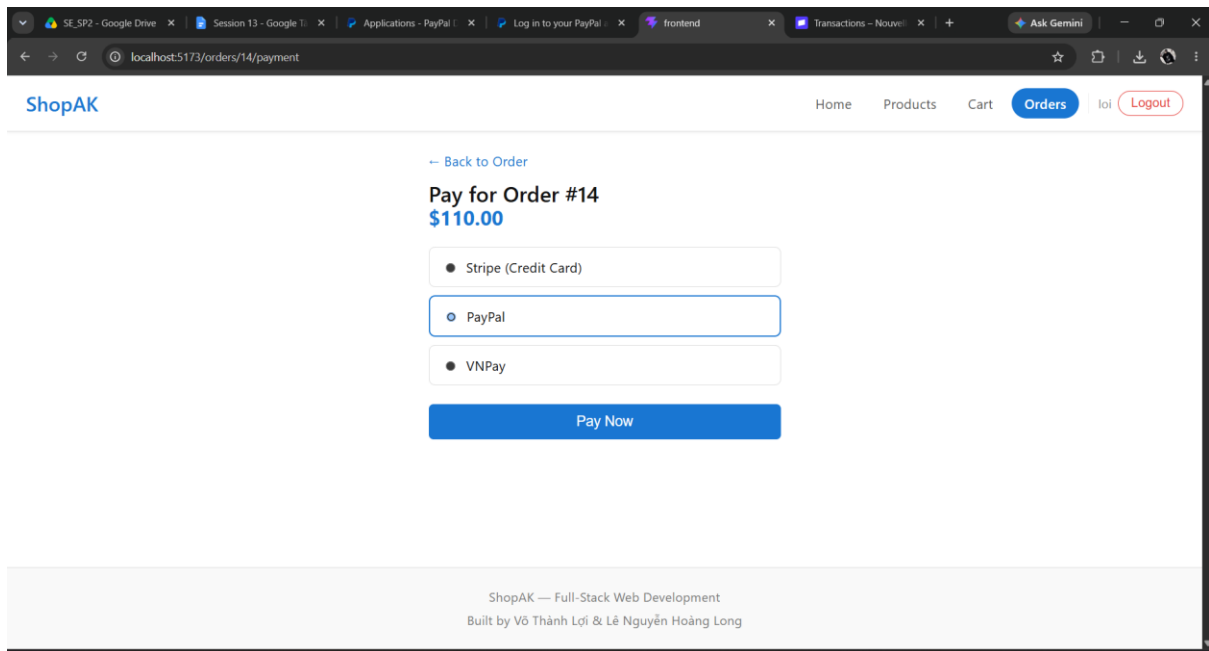


Scenario 2 – PayPal

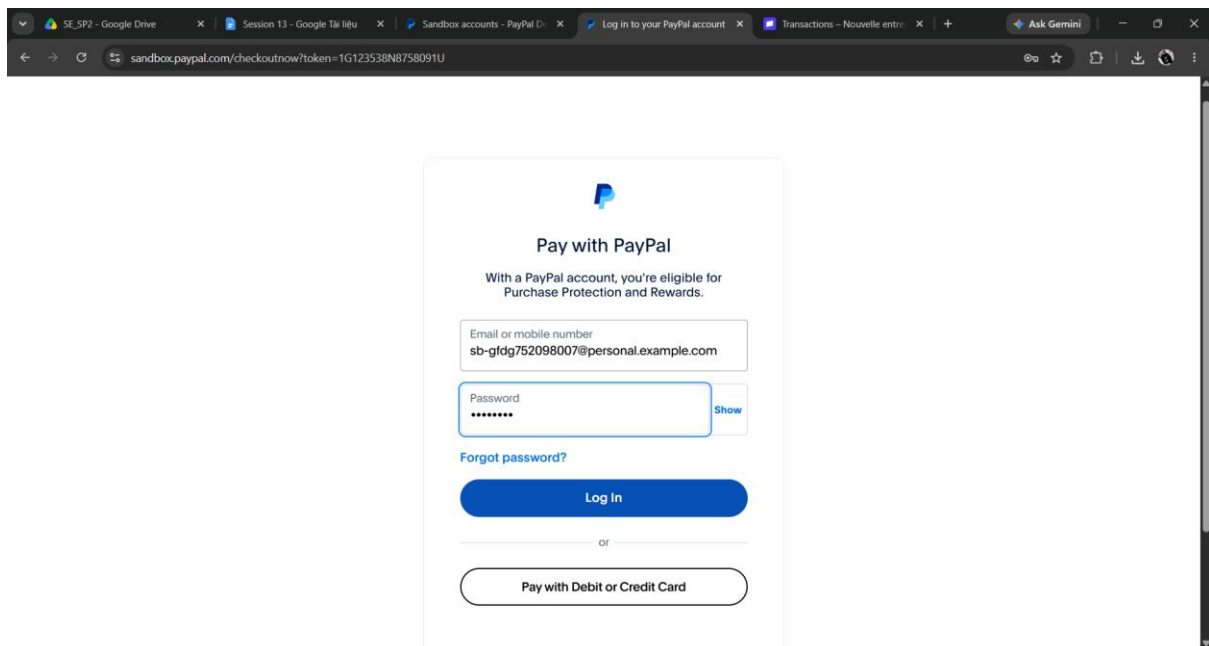
Tạo đơn mới khác vào payment page chọn PayPal và nhấn Pay Now để thanh toán (Order #14)



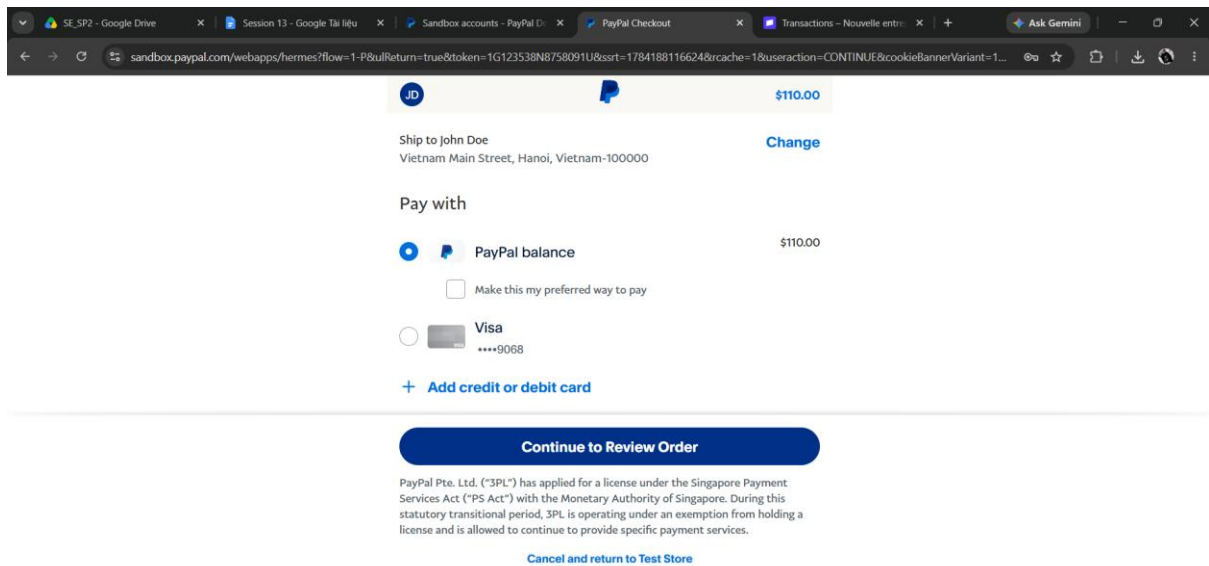
Sau khi nhấn paynow sẽ redirect tới PayPal sandbox



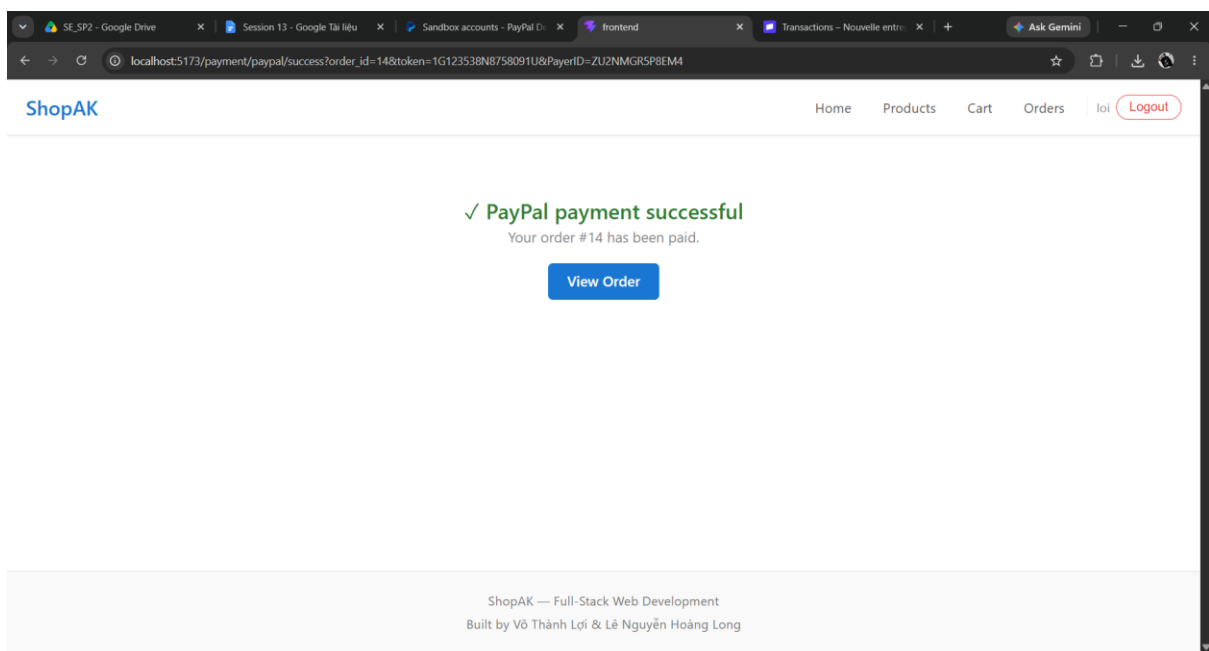
Đăng nhập bằng tài khoản **Personal sandbox** đã tạo



xác nhận thanh toán PayPal



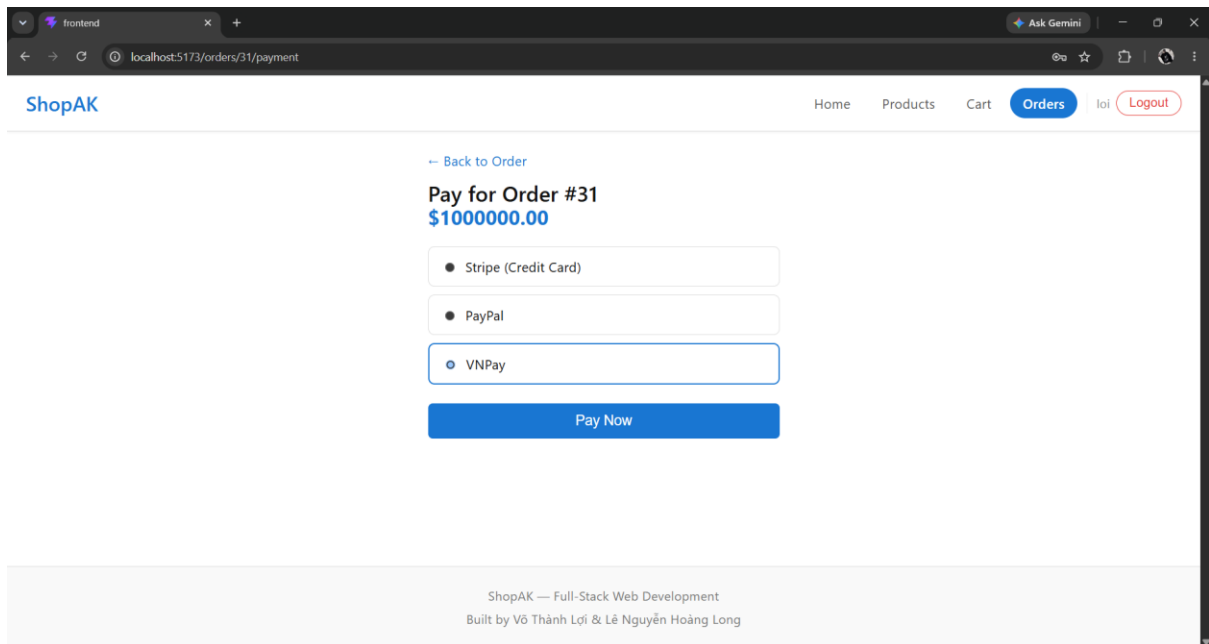
redirect về success page



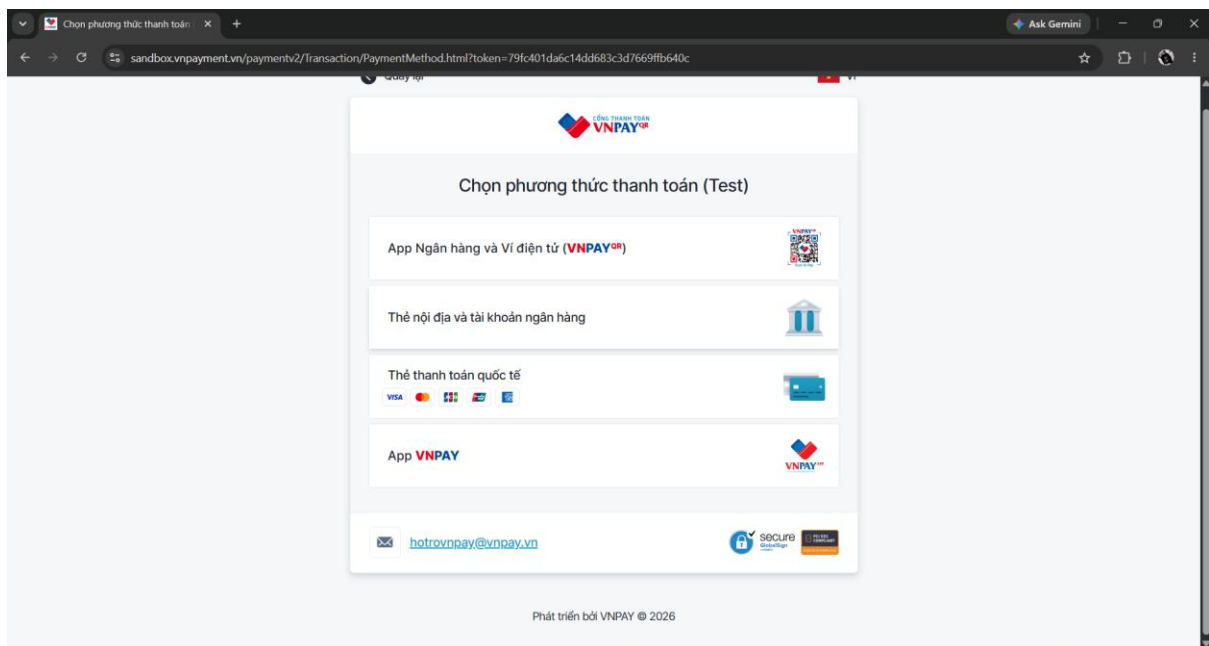
Trong PayPal Sandbox Dashboard (Activity/Transactions) sẽ thấy giao dịch vừa thực hiện

Scenario 3 – Vnpay

Tạo đơn mới chọn VNPay và nhấn Pay Now



Chọn thẻ nội địa và nhập vào ngân hàng NCB



Sau khi nhập ngân hàng NCB sẽ hiện ra form giao dịch sau đó nhập số thẻ vào và nhập OTP xác nhận thanh toán

